

Viva Question Bank, with Answers

AI-Powered Legal Document Analysis System: Clause Detection, Category-Level Risk
Prioritization and Plain-Language Explanation on CUAD

Group ID P25301051 Jerin Aktar Afnan Mazumdar Shoyeb Hasan Sayem
Supervisor: Utsha Kumar Roy
BRAC University, Department of Computer Science and Engineering

Contents

How to use this document	1
Part A , The eight questions	3
Part B , Dataset and preprocessing questions	12
Part C , Model and method questions	14
Part D , Training questions	16
Part E , Results and evaluation questions	18
Part F , Risk taxonomy questions	20
Part G , Summarizer questions	21
Part H , Clausify application questions	22
Part I , Ethics, limitations and threats to validity	24
Part J , Hard and hostile questions	25
Part K , Number sheet	27

How to use this document

Part A answers the eight questions you asked for, in full. Parts B to J are the wider question bank, everything an examiner can reasonably ask from the report and from the Clausify application, grouped by topic.

Each answer has two layers:

- **Say this** , the 20 to 40 second spoken answer. This is what you deliver.
- **If pushed** , the follow-up detail, for when the examiner says “and why?” or “prove it”.

Every number in here is taken from the submitted report and the result files in **analysis/**. Nothing is invented. Part K is a one-page number sheet to memorise.

The single most important habit in a viva: **give the number, then the reason**. Examiners are testing whether you know your own system, not whether it is perfect. This project’s strength is that it measured its

own failures, so saying “it does not work, and here is the measurement that shows why” is a winning answer, not a losing one.

Part A , The eight questions

A1. Where did we find the dataset?

Say this: We used CUAD, the Contract Understanding Atticus Dataset. It was built by The Atticus Project, a non-profit of contract lawyers, and published by Hendrycks and colleagues at the NeurIPS 2021 Datasets and Benchmarks track. We downloaded the official release from the Hugging Face hub, repository `theatticusproject/cuad`. It has 510 real commercial contracts taken from public US SEC filings, and experienced attorneys highlighted every clause belonging to 41 categories. The annotation is reported to be worth around two million US dollars of attorney time, which is exactly why we could not have built anything like it ourselves.

If pushed , the practical detail that shows you actually handled the data: The default `load_dataset()` call on that repository is a trap. It gives you only the 511 raw contract PDFs, which you cannot train on. The annotated data lives in two files inside the repository and we used both for different purposes:

- `master_clauses.csv` , a wide table, 83 columns, one row per contract. We used it for exploratory analysis and for building the presence labels.
- `CUAD_v1.json` , the SQuAD-format file with full contract text and character-level answer offsets. We used it for span-extraction training.

From the CSV we did a wide-to-long melt, 41 categories times two columns each, which gave $510 \times 41 = 20,910$ labelled (contract, category) rows, 32.1 percent of them positive. Two decisions there were not obvious. First, we paired the clause-text column with its answer column **by position, not by name**, because one header, “Notice Period To Terminate Renewal- Answer”, has an inconsistent space that silently breaks string matching. Second, we defined a category as present when the clause-text list is non-empty, rather than reading the Yes/No answer column, because eight categories are entity-style, dates and party names, and their answer column holds a value instead of Yes or No.

Likely follow-up , “why not collect your own contracts?” Legal annotation needs lawyers, and lawyers are expensive. That is the whole reason the field was blocked before CUAD existed. We are three CS students with a laptop and one semester. The honest answer is that building even a 20-contract annotated set to this standard was outside our means, and that is also why our risk taxonomy, the one part we did author ourselves, carries a validation warning throughout the report.

A2. Which models did we use?

Say this: Five components, three of them trained. In pipeline order:

1. **TF-IDF plus logistic regression** , the presence baseline. 20,000 features, word uni- and bi-grams, and one class-balanced logistic regression per category, 41 of them.
2. **DistilBERT-base, sequence classification head** , the windowed presence model. It scores a (question, window) pair and we max-pool across the windows of a contract.
3. **A parameter-free ensemble** of those two, with three rules: AND which is the minimum of the two probabilities, OR which is the maximum, and AVG which is the mean.
4. **DistilBERT-base, question-answering head** , the span extractor. It predicts start and end token positions for the clause inside a window.
5. **FLAN-T5-small** , the plain-language stage, fine-tuned on 41 hand-written category templates.

On top of those sits the **risk layer**, which is deliberately not a model. It is a fixed lookup table, 41 categories mapped to High, Medium or Low, 8 High, 22 Medium, 11 Low.

If pushed , “why so small? DistilBERT is not state of the art.” Correct, and we say so in the report. Everything was trained on one Apple M4 laptop with 16 GB of unified memory. Our first-choice backbone was DeBERTa-v3, which is what the CUAD paper uses, but it produced NaN gradients on Apple’s MPS backend and ran at about 89 seconds per step on CPU, which is unusable. We switched to DistilBERT, 67 million parameters. The method does not change, only the backbone string, so on CUDA hardware it can be swapped straight back. Our absolute scores understate what the method would do at scale, and we never compare our span numbers to the published CUAD numbers because theirs use DeBERTa-v3-large, roughly five times our parameter count.

A3. How did we implement those models?

Say this , walk the pipeline:

Stage 2A, presence. The contract is sliced into 2,000-character windows advancing in 1,500-character steps, so neighbours overlap by 500 characters. For each of the 41 categories we pair the category’s CUAD question with every window and the classifier predicts whether that window contains a clause of that category. The document-level score is the **maximum over all windows**. That is multiple-instance learning: the document is a bag, the windows are instances, and “the clause is somewhere in this contract” is exactly “at least one window is positive”. Training used 24,000 of the 53,662 window examples, stratified to hold the positive rate at 39.2 percent, two epochs, batch 16, learning rate 2e-5, on MPS, about 50 minutes.

The baseline in parallel. TF-IDF has no length limit, so it reads the *whole* contract, which no windowed transformer can. Fitted on the 408 training contracts, one logistic regression per category, trains in under a minute.

The ensemble. We combine the two with min, max and mean of the probabilities. Nothing in those rules is tunable, which is deliberate, because a parameter-free rule cannot overfit the test set.

Stage 2B, span. `DistilBertForQuestionAnswering`, input is (CUAD question, window), output is start and end token positions. Character offsets from `CUAD_v1.json` are converted to token labels through the tokenizer’s offset mapping. 8,000 of 11,156 examples, 320 tokens, lr 3e-5, about 23 minutes.

Stage 3, plain language. We wrote 41 gold templates, one sentence per category from the weaker party’s point of view, paired every extracted clause with its category’s template, which gives 11,156 pairs, and fine-tuned FLAN-T5-small on the prompt `summarize in plain English: <clause>`. 4,000 examples, lr 3e-4, batch 8, on CPU, about 8 minutes.

If pushed , “why are there two different window sizes? That looks inconsistent.” This is a favourite examiner question and it has a clean answer. They do different jobs. The 2,000-character window is a **presence** window: it tiles the document blindly, because at that point we do not know where anything is. The 1,200-character window in Stage 2B is a **focus** window: it is not tiled, it is re-centred so the gold answer begins about 150 characters in. The reason is purely mechanical, a 1,200-character window fits inside the model’s 320-token limit so the target span survives truncation, whereas an answer sitting at the tail of a 2,000-character window would simply be cut off and we would be training on an example whose answer is not there.

The leakage question hiding inside that , be ready for it. Re-centring uses the gold offset, which is a label. That is legitimate only because it happens at **training time only**. It decides which text becomes a training example; it never decides what the model reads at inference. At inference there is no gold offset and Stage 2B just runs over whatever window Stage 2A selected. If we re-centred at inference the evaluation would be meaningless. Our general rule across the whole project was: labels may shape training targets, they may never shape an input a model sees or an inference-time selection.

A4. Specifically, why did we use these models?

Answer this one model by model. Each has a different reason, and that is the point.

Why TF-IDF plus logistic regression? Two reasons. First, legal drafting is extremely formulaic, “This Agreement shall be governed by the laws of the State of ...”, so distinctive vocabulary alone carries a great deal of signal. Second, and more important, **a bag-of-words model has no input length limit, so it reads the entire contract.** That is an advantage no windowed transformer has. We built it as the bar every neural model had to clear, and it is the component that ended up winning.

Why a windowed transformer, and why multiple-instance learning rather than retrieval? Because we measured the alternative first. The obvious design is retrieval: score windows by similarity to the category query and hand the model only the best few. Before training anything we measured the **retrieval hit-rate**, how often the selected window actually contains the gold clause. The best strategy, category name as query with top-3 windows, reached only **64.0 percent**. That is a hard ceiling on downstream recall, because a clause in a window the retriever never selects cannot be recovered by any model after it. 64 percent sits below the baseline’s 0.775, so a retrieval-based transformer was going to lose before it was ever trained. The measurement took seconds and saved a wasted training run. So we removed the retriever and classified every window, with max-pooling as the aggregation rule.

Why DistilBERT specifically? Three reasons, in order of force: DeBERTa-v3 produced NaN gradients on MPS; DistilBERT at 67M parameters trains in under an hour on the laptop we had; and Clausify has to answer in seconds, which a distilled model can do and a large one cannot.

Why not Longformer or another long-context model? We benchmarked it rather than asserting it. Longformer-base, 148.7M parameters at 4,096 tokens on our M4: a single training step takes **258 seconds** and 14.71 GB at batch size 1. Reproducing our two-epoch presence schedule would take roughly **136 days** of continuous compute. The DistilBERT run it replaces took 47 minutes. On the inference side, scanning one median contract for all 41 categories is about **10.6 seconds** with DistilBERT against **454 seconds**, seven and a half minutes, with Longformer. Be honest about the part we got wrong: our report originally claimed Longformer would not *fit* in memory. It does, at batch size 1, 14.71 GB on a 16 GB machine. We measured it, found we were wrong, and corrected the claim in the report rather than quietly rewriting it. The binding constraint is time, not memory.

Why FLAN-T5-small? It is instruction-tuned, so it responds to a natural-language prompt without architecture changes, and it is small enough to fine-tune on CPU in 8 minutes. We also report, honestly, that this stage did not work as a summarizer, see A6 and Part G.

Why a lookup table for risk rather than a learned model? Because it can be audited line by line, its reasoning is written down per category, and it cannot drift silently between versions. We had no risk-labelled data to learn from anyway. The cost of that choice is that it cannot tell a ten-thousand-dollar liability cap from a ten-million-dollar one.

A5. How do the models work?

This is the “explain the mechanism” question. Keep it concrete.

The TF-IDF baseline. Every contract becomes a 20,000-dimensional vector where each entry is the weighted frequency of a word or word-pair, down-weighted by how common that term is across the corpus. A logistic regression per category learns a weight per term and outputs a probability. Class weights are balanced so the rare categories are not simply predicted absent. There is no notion of word order or meaning, only which terms appear and how distinctively.

The windowed DistilBERT, and the MIL rule. DistilBERT is a 6-layer transformer, a distilled BERT, that reads a pair of texts and produces a single vector for the pair, which a classification head turns into two logits, present or absent. We feed it (category question, window). A median contract produces about 22 windows, so for 41 categories that is roughly 900 forward passes per contract. Then the aggregation:

$$\text{score}(\text{contract}, \text{category}) = \max \text{ over all windows of } P(\text{window contains this clause})$$

The reason max is the right rule in principle: “this contract contains a non-compete” is true when **at least one** window contains it, which is exactly the multiple-instance assumption. The reason it hurts in practice, and this is the insight an examiner will like: **a maximum over 35 windows is 35 chances to be wrong.** Every spurious window inflates the document score, so recall goes up to 0.908 and precision falls to 0.562, and the score distribution piles up near 1, which is why a 0.5 decision boundary is badly placed for it.

The 500-character overlap. It is not arbitrary. The median annotated clause is 196 characters. The overlap has to exceed the clause length, otherwise a clause could straddle the boundary of two windows and appear complete in neither. 500 comfortably exceeds 196, so every clause lands wholly inside at least one window.

The ensemble. The two models are complementary: the transformer wins on semantically subtle categories such as No-Solicit of Employees and Liquidated Damages, the baseline wins on keyword-heavy ones. AND takes the minimum of the two probabilities, so a clause is reported only when both models agree. For a false positive to survive, the lexical model and the transformer must make the **same** mistake on the **same** contract and category, and their mistakes are largely independent. That is what recovers the precision max-pooling loses: precision goes from 0.562 to 0.776.

The span model. Same backbone, different head. Instead of one classification output it produces two distributions over token positions, a start distribution and an end distribution. We take the argmax of each inside the context portion, decode the tokens between them, and the product of the two softmax probabilities is the confidence. We cap the span at 80 tokens so a runaway prediction cannot swallow the page.

The summarizer. A sequence-to-sequence model. The prompt and clause go in, a sentence comes out token by token. What it actually learned is covered in A6.

The risk layer. Pure dictionary lookup. Category name in, (level, one-line reason) out, then the detected clauses are sorted High, Medium, Low and by confidence within each band.

A6. Future work, in short

Lead with the two that need no compute, because they are the two real gaps.

Say this: Two things first, and neither needs a GPU.

1. **Get the risk taxonomy reviewed by someone with legal training.** It is the main thing we add on top of CUAD and it is the component with the weakest backing, three CS students wrote it. The review instrument is already in the repository, `review/risk_taxonomy_review_sheet.csv`, one row per category. Two or three reviewers, half an hour each, then `review/agreement.py` computes Cohen's or Fleiss' kappa and lists the disputed categories. It is the cheapest credibility this project can buy.
2. **Observe someone using Clausify.** Five to ten participants reviewing a contract with and without the tool, measuring task time, clauses correctly identified, and above all whether users **over-trust** the output. We claim a risk-prioritized report helps a non-lawyer and we have no evidence for it.

Then the technical list:

3. **Select on the objective that matters.** Carve a validation split from the 408 training contracts, choose the ensemble rule and thresholds on a risk-weighted criterion rather than micro-F1, and report once on the test set.
4. **Calibrate the confidence score.** Platt scaling or isotonic regression on a validation split. Expected Calibration Error is currently 0.201. No retraining needed.
5. **Give the span model a way to abstain**, SQuAD v2 style unanswerable examples, so it stops answering confidently from windows that cannot contain the clause.
6. **Train the span model on the windows it will actually see**, by varying the answer offset instead of re-centring every example at 150 characters.
7. **Evaluate summaries for faithfulness, not overlap**, BERTScore, an entailment check, a readability statistic, and a small human rating of whether legal meaning survives.
8. **Attention pooling over windows**, a learned aggregator rather than a fixed statistic.
9. **Cross-validation instead of one split**, since split variance was the largest uncertainty we measured.
10. **Dense retrieval.** Our 64 percent ceiling was measured with TF-IDF scoring only. A sentence-encoder retriever matches on meaning and may raise it enough to make retrieve-then-read competitive.
11. **A trained Longformer baseline on CUDA**, which is the honest comparison our cost argument currently substitutes reasoning for.
12. **Scale the backbones**, DeBERTa-v3 and FLAN-T5-base on CUDA, to close the gap to published CUAD results.
13. **A genuinely abstractive summarizer**, by growing the 41-template seed into a few thousand clause-specific targets. Our 150-clause pilot is a working prototype of that annotation protocol.
14. **Cross-clause conflict detection**, for example Exclusivity against Most Favored Nation, which we designed in the pre-thesis phase but never deployed.
15. **Instance-level risk**, reading the span itself, a cap's actual amount, a non-compete's actual duration, instead of a per-category level.
16. **Contracts from outside CUAD**, and specifically Bangladeshi contracts, which is the deployment context that motivated the work and on which we have zero evidence.

If pushed , “which one first, and why?” The taxonomy review. Every other component in the thesis is measured against gold annotations that experts produced. That one is measured against nothing, and it has the most direct route to the user, the level decides what the app tells a signer to worry about first. A category we marked Medium that a practitioner would mark High is a warning that never appears.

A7. Industrial use of the project

Be careful here. Claim the use case, not the readiness.

Say this: The target user is the party who signs a contract without a lawyer reading it first, which in practice means small businesses, startups, freelancers and individuals. The industrial shape of that is four things:

1. **First-pass triage before signing.** A small business receives a 40-page distribution agreement. The system tells them in under a minute which of 41 clause types are in it and which three or four deserve a lawyer’s half-hour. That converts an unaffordable full review into an affordable targeted one. It does not replace the lawyer, it tells you which page to put in front of them.
2. **Due-diligence support inside law firms and in-house teams.** During an acquisition a team may review hundreds of contracts for the same handful of provisions. CUAD exists precisely because that is what lawyers do. A clause-presence pass over a document set, with the risky categories surfaced first, is a filtering step, with a human verifying every output.
3. **Procurement and contract-lifecycle management.** Organisations that sign many supplier contracts can run an automatic flag for the categories they care about, auto-renewals, uncapped liability, exclusivity, minimum commitments, and route the flagged ones to review.
4. **Legal-access tooling in markets with few affordable lawyers,** which is the Bangladeshi context that motivated the project.

The deployment property worth selling, and it is genuine: Clausify runs entirely locally on small models, CPU only, median 27 seconds per contract and about 1.5 GB of memory. Contracts are among the most commercially sensitive documents an organisation holds, and our pipeline never transmits one to a third party. That is a real industrial advantage over an API-based tool, and it is one of the few places where our laptop-budget constraint produced the better design. Be precise though: the privacy comes from the deployment topology, running locally, not from anything clever the application does. Hosted on a shared server, the same code would put the user’s contract in the memory of a machine they do not control.

The honesty clause , say this unprompted, it strengthens you: We would not deploy this today. Three blockers. The quoted clause text is unreliable end to end, only about 22 percent survival, so the product claim has to be “a clause of this type appears to be here, look” and not “this is the clause”. The confidence bar is miscalibrated, ECE 0.201. And the risk levels have never been reviewed by a lawyer. The category name and the risk badge are the trustworthy parts; the sentence underneath is not yet. A commercial version has to fix those three before it ships, and none of them need a bigger model.

Likely follow-up , “who are the competitors?” Commercial contract-review tools exist, Kira, Lumiance, eBrevia, and law firms use them in due diligence. They are expensive, enterprise-priced, cloud-based, and aimed at legal professionals reviewing documents, not at a non-lawyer deciding whether to sign. Our differentiator is the output format, risk-ranked and explained for a signer, and local execution. Our disadvantage is every other dimension, accuracy, coverage, jurisdiction, and validation.

A8. Is there a dataset more advanced than CUAD? If yes, why did we not use it?

Say this: Yes, several newer legal datasets exist, but none of them replaces CUAD for this task. The distinction that matters is what the annotation actually gives you.

What CUAD uniquely provides: attorney-annotated, **character-level spans** for **41 clause categories**, over **entire unsegmented commercial contracts**. For a system that has to both decide *whether* a clause is present and point at *where* it is, that combination is still the reference dataset.

The main alternatives, and why each one does not fit:

Dataset	What it is	Why we did not use it
MAUD (Atticus, EMNLP 2023)	47,457 expert annotations over 152 merger agreements , 92 questions from the ABA Deal Points Study	Newer and arguably more expert-intensive, but a single contract type and a multiple-choice reading-comprehension format, not 41-category span detection over general commercial contracts
ContractNLI (Koreeda and Manning, EMNLP Findings 2021)	607 NDAs , 17 fixed hypotheses, entailed / contradicted / not mentioned, plus evidence spans	NDAs only, and the task is document-level inference against 17 hypotheses, not clause-category extraction. Useful as a future evaluation, not as a substitute
LEDGAR (LREC 2020)	Hundreds of thousands of contract provisions with a very large label set	Much bigger, but the provisions are already segmented . It removes exactly the hard part of our problem, finding the clause inside a 33,000-character document
LexGLUE (ACL 2022)	A 7-task benchmark suite for legal language understanding	A benchmark suite, not a contract-clause corpus. Its tasks are judgement prediction, statutory classification and similar, none of which is span extraction over contracts
LegalBench (2023)	A large collaborative suite of legal reasoning tasks for evaluating LLMs	An evaluation harness for large models, with no training corpus of the kind we needed, and we had no budget to run large models anyway
Unfair-ToS / CLAUDETTE	Terms-of-service clauses labelled for potential unfairness	The closest public thing to a <i>risk</i> label, which is exactly what we lacked, but it is consumer terms of service rather than negotiated commercial contracts, and a small corpus

The honest extra point, and it is the stronger version of the answer: The dataset gap that actually constrained this project was not that CUAD is old. It is that **no public dataset labels clause severity**

at all. CUAD tells you a non-compete is present; it does not tell you how dangerous that particular non-compete is for the signer. LexGLUE and the rest operationalise risk indirectly, as classification into legally meaningful categories, never as a severity judgement. That absence is why we had to author the 41-level taxonomy ourselves, and it is why that taxonomy is the part of the thesis with the weakest evidential backing. If a severity-labelled contract corpus existed, we would have used it and the weakest part of this work would disappear.

If pushed , “would a bigger dataset have fixed your transformer’s defeat?” Not on the evidence we have. We tested the budget explanation directly: retraining the presence model on all 53,662 windows instead of 24,000 moved micro-F1 by **minus 0.0004**. Doubling the data within CUAD changed nothing. That does not prove a ten-times-larger corpus would not help, but it does mean “we just needed more data” is not an explanation we are entitled to give.

Part B , Dataset and preprocessing questions

B1. How many contracts, categories, and labelled examples? 510 contracts, 41 categories, 20,910 (contract, category) rows, 32.1 percent positive. 408 contracts train, 102 test.

B2. Why 510 and not 511? The repository ships 511 raw PDFs; the annotated master CSV covers 510 contracts. We work from the annotations, so 510.

B3. How did you split the data, and why that way? 80/20 at the contract level, fixed seed 42, so whole contracts go to one side. If we had split at the clause or row level, clauses from the same contract would appear on both sides and the model could memorise a contract’s drafting style. The same split is reused identically by all three models, so “test” always means the same 102 contracts.

B4. Is the split balanced? Aggregate positive rates are 32.0 percent train against 32.2 percent test, but a single pair of percentages can hide a skew, so we checked all 41 categories. Pearson $r = 0.986$, mean absolute difference 3.4 percentage points. One real outlier: Termination For Convenience is in 47.1 percent of test contracts against 33.1 percent of training contracts, a 14-point gap. We report that rather than redrawing the split, because redrawing until it looks good is a form of test-set tuning.

B5. Is your split a lucky one? No, and we measured it. Across 20 random splits the baseline averages 0.7829 with SD 0.0063. Our split gives 0.7747, which sits **below** the mean. If anything we report the baseline at a slightly unfavourable draw.

B6. What is seed 42, and why that number? A seed is the starting value for the pseudo-random number generator. Every “random” operation in the project, drawing the 80/20 split, shuffling batches, initialising the classification head, sampling the 150 summarizer clauses, is really deterministic given the seed, so fixing it makes the whole project reproducible: anyone running `presence_run.py` gets our exact split and our exact numbers. We used one global seed, 42, for the split, all three trainings and every bootstrap.

The number itself is arbitrary and that is the point. 42 is the conventional default in ML code, a joke borrowed from *The Hitchhiker’s Guide to the Galaxy*. It carries no statistical meaning. The honest way to say it: **we needed a fixed seed, and we picked the conventional one rather than searching for a good one.** If we had tried several seeds and kept the one with the best score, that would be a form of test-set tuning and every number in the thesis would be optimistic.

If pushed , “how do we know 42 was not a lucky draw?” We measured it, twice, from two directions. *Split luck:* we redrew the split 20 times. The baseline averages 0.7829 with SD 0.0063, range 0.7702 to 0.7918. Our seed-42 split gives **0.7747**, which sits **below** the mean, in the lower part of the distribution. If anything we report the baseline at a slightly unfavourable draw. *Training luck:* we retrained the presence model at seeds 42, 43 and 44. Micro-F1 0.6943, 0.6928, 0.7082, SD 0.0085. The baseline’s lead of 0.0805 is 9.5 times that SD, and even the luckiest seed still loses by 0.0665. There is no seed at which the transformer becomes competitive.

Dataset-split variance, SD 0.0063, turned out to be the **largest** of the three uncertainty sources we measured, larger than model seed and larger than the bootstrap half-width, which is why proper cross-validation rather than one split is in future work.

B7. How long are the contracts? Why does it matter? Median 33,143 characters, about 5,000 words; longest 338,211. That is roughly 16 times a transformer’s effective input window. This single fact is the central modelling tension of the whole thesis, every design decision follows from it.

B8. How long are the clauses? Median 196 characters, 90th percentile 587, 99th percentile 1,330. So a clause fits easily inside one window. The hard part is not reading the clause, it is finding it in a very long document.

B9. How bad is the class imbalance? Severe and very uneven. Document Name is present in 100 percent of contracts, Parties 99.8 percent. At the other end, Source Code Escrow is 2.5 percent, 13 positive contracts in the entire corpus, and Price Restrictions 2.9 percent. The median category appears in about 23 percent of contracts.

B10. How did you handle the imbalance? Three ways. Class-balanced logistic regression for the baseline. Two negative windows per positive bag for the transformer, holding the training positive rate near 39 percent. And in reporting, we give macro-F1 both over all 41 categories (0.572) and over the 34 with at least ten test positives (0.666), so the rare tail does not silently drag the headline.

B11. What preprocessing did you do to the text? Very little, deliberately. The contract text is used as-is, UTF-8, and sliced into windows. No lowercasing, no stopword removal, no lemmatisation for the transformer path, because WordPiece handles that. TF-IDF uses sublinear term frequency and `min_df = 2`.

B12. Did you do any data augmentation? No. With a dataset this small it is tempting, but augmenting legal text risks changing the legal meaning of a clause, which would corrupt the label.

B13. What is a (contract, category) row exactly? One decision: “does contract X contain a clause of category Y?”. $510 \times 41 = 20,910$ such decisions, 4,182 of them in the test set. That is the unit everything in the presence evaluation is counted over.

Part C , Model and method questions

C1. What is multiple-instance learning, in one sentence? Learning when labels are attached to a bag of instances rather than to individual instances; here the contract is the bag, the windows are instances, and a positive bag means at least one window is positive.

C2. Why max-pooling and not mean-pooling? Because max is the multiple-instance decision rule, mean is not. “The contract contains a non-compete” is true if any window contains it, so the aggregate should be a max. We did also test alternatives after the fact, see C3.

C3. Did you test other aggregation rules? Yes, six of them, each at its own best threshold: max, top-3 mean, top-5 mean, mean over all windows, noisy-OR, and second-highest window. Two findings. At the 0.5 threshold we actually used, max was **not** the best choice, top-3 mean scores 0.7144 against max’s 0.6943. But once each rule gets its own threshold, max is the best of the six at 0.7504. And critically: **no aggregation rule at any threshold reaches the baseline’s 0.7747**. That closes off aggregation as an explanation for the transformer’s defeat.

C4. Why is mean-over-all-windows so bad at 0.5? Recall collapses to 0.091, for an obvious reason: an average over 35 windows of which one is positive cannot reach 0.5. Its best threshold is 0.10, which is the diagnostic, not the fix.

C5. Why 2,000 characters and 1,500 stride specifically? Not tuned. The constraint is that the overlap must exceed the median clause length so no clause falls between two windows. Median clause is 196 characters, overlap is 500. Any pair satisfying that would do; we did not search for the best one, and we say so.

C6. Why 256 tokens for the presence model when the window is 2,000 characters? 2,000 characters is roughly 400 to 500 tokens, so 256 truncates. That is acceptable for presence because the task is “does this region look like a non-compete”, a judgement the first part of the window usually supports, and the 500-character overlap means the truncated tail of one window is the head of the next. For the span model, where the exact boundary matters, we use 320 tokens over a smaller 1,200-character focus window precisely so nothing is cut.

C7. Why is the ensemble parameter-free? Is that not leaving performance on the table? Yes, deliberately. Min, max and mean have nothing to tune, so no choice we made can have been fitted to the test set. A weighted combination would almost certainly score higher, and it would also be a number we could not defend, because we have no validation split carved out to fit the weight on. That is in future work.

C8. Why does AND beat both of its own components? Because the two models fail differently. The transformer over-detects, precision 0.562, recall 0.908. The baseline is more balanced, 0.742 and 0.810. Requiring agreement removes false positives that only one model makes, and since their mistakes are largely uncorrelated, most false positives are removed while most true positives survive. On the 31 common categories the effect is clean: against the transformer alone the ensemble gains **19.4 points of precision for 12.1 points of recall**, precision 0.582 to 0.776 and recall 0.930 to 0.810, and F1 rises from 0.716 to 0.792. That group is where the ensemble’s entire aggregate advantage is generated.

C9. Is the ensemble’s win real? No, and this is the kind of answer that earns marks. We ran a cluster bootstrap, resampling **contracts** rather than individual decisions, 10,000 resamples. The difference interval is [-0.0068, +0.0089], it straddles zero, and the ensemble is ahead in only 62.6 percent of resamples. The two are statistically indistinguishable on this test set. We report the top two rows of our own leaderboard as a tie.

C10. Why cluster bootstrap and not a normal one? Because the 4,182 test decisions are not independent. They come from 102 contracts, and 41 decisions share a contract, so they are correlated. Resampling individual rows would understate the uncertainty. We saw the effect directly on the span numbers: the contract-clustered interval on token-F1 is [0.740, 0.782], roughly **twice as wide** as the [0.751, 0.774] you get by pretending clauses are independent.

C11. Why did you not use an LLM, like GPT-4 or Claude? Three reasons, in order. Cost and

reproducibility, we needed every number regenerable from files on disk. Privacy, the deployment story depends on the contract never leaving the machine. And scope, the thesis question was whether a small, local, trainable pipeline can do this, not whether a frontier model can. We did use an LLM assistant for one narrow, declared purpose: drafting the 150 clause-specific reference summaries in the summarizer pilot, written from clause text alone with no sight of any model output, and we label them a pilot reference set, not annotation.

C12. What is DistilBERT and how does it differ from BERT? A 6-layer distilled version of BERT-base, about 67 million parameters against 110 million, trained to reproduce BERT’s outputs. Roughly 60 percent faster, with most of the accuracy on standard tasks.

C13. Why DistilBERT for both presence and span? Could you share one model? Same backbone, different heads, but they are two separately fine-tuned checkpoints because the tasks are different, sequence classification against start/end position prediction. A single multi-task model is possible and would halve the app’s memory, but we did not try it.

Part D , Training questions

D1. What hardware? One Apple M4 laptop, 16 GB unified memory, MPS (Metal) backend, torch 2.8, transformers 5.x. Presence and span on MPS, summarizer on CPU.

D2. How long did training take? Presence about 49 to 52 minutes, span about 23 minutes, summarizer about 8 minutes. The whole pipeline retrains in under 90 minutes.

D3. What were the hyperparameters, and how did you choose them? Presence: 24,000 examples, 256 tokens, 2 epochs, batch 16, lr 2e-5. Span: 8,000 examples, 320 tokens, 2 epochs, batch 16, lr 3e-5. Summarizer: 4,000 examples, 256 in / 48 out, 2 epochs, batch 8, lr 3e-4. **None of them were tuned.** They are the published fine-tuning defaults for DistilBERT and T5-family models. We ran each configuration once and never compared variants on the test set.

D4. Is “we did not tune” a strength or a weakness? Both, and say both. It is a genuine guarantee that no setting was chosen by scoring on the test set, so the held-out numbers are honest. It is also a limitation: the transformer competes at conventional defaults against a baseline that has essentially no hyperparameters to get wrong. The threshold result proves this is not hypothetical, an un-chosen 0.5 costs the transformer 0.056 micro-F1. We list it as the most serious internal-validity threat in the report.

D5. What optimizer and schedule? AdamW, linear decay, no warmup, gradient-norm clipping at 1.0, for all three models. The presence and span settings were read back from the saved `training_args.bin` rather than from our notes, so what we plot is what actually ran.

D6. No warmup, is that safe? Mostly, for short fine-tuning runs of 1,000 to 3,000 steps starting from pre-trained weights at small learning rates. But there is a caveat we discovered and reported: across 63 logged points the gradient norm ranges 0.55 to 11.97, median 4.66, and **98.4 percent of logged steps exceed the clipping threshold of 1.0**. Nothing diverged, no NaNs, loss fell smoothly. But clipping here is not a rare safety net, it is active on essentially every step, so part of what kept early training stable was the clipping rather than the small learning rate. We would not want anyone to read “no warmup was fine” as a general recommendation on the strength of our smooth curves alone.

D7. What validation did you use? Per model, and this is the table an examiner may ask for directly. Presence: 5 percent of the `training` windows, split off before training, used for monitoring only, never for checkpoint selection. Span: none, `eval_strategy="no"`. Summarizer (the deployed checkpoint): none. All three used `save_strategy="no"` and kept final-step weights.

D8. Did you ever touch the test set during development? Once, wrongly, and we report it. An exploratory notebook run of the summarizer used 300 clauses drawn from the `test` split as its evaluation set. That is the wrong data to monitor on. Two things limit the damage and we state them without treating either as an excuse: the run had `save_strategy="no"` and no early stopping, so nothing could be selected from it; and the deployed summarizer was rebuilt with no evaluation set at all, and it is that checkpoint that produced every reported number. So no reported score was computed by a model selected using test data. We left the figure in the report with the correction attached rather than quietly regenerating a clean one.

D9. Did you measure seed variance? For the presence model, yes, three seeds. Transformer micro-F1 0.6943, 0.6928, 0.7082, mean 0.6984, SD 0.0085. AND ensemble SD 0.0022. The baseline is deterministic. Span and summarizer are single runs and carry an unmeasured variance component, which we state as a limitation.

D10. What about overfitting? Presence train loss fell 0.320 to 0.261 and validation 0.278 to 0.257 across two epochs, with the curves staying close together, so no sign of overfitting at this scale. More broadly, the guard is structural: 510 contracts is the entire dataset, so every conclusion rests on held-out numbers from contracts no model saw.

D11. What went wrong during training? Four things, all documented. DeBERTa-v3 produced NaN gradients on MPS and ran at about 89 seconds per step on CPU, so we switched backbone. MPS memory does not release in-process, so after two trainings the summarizer hit the ceiling and we moved it to CPU.

transformers 5.x renamed `Trainer(tokenizer=...)` to `processing_class=...`, and we disabled mixed precision and cast loss logits to fp32 for MPS dtype safety.

Part E , Results and evaluation questions

E1. What are your headline numbers? 85.5 percent accuracy and micro-F1 0.776 for clause presence on the 4,182 held-out decisions, and token-F1 0.763 for span extraction given the right window.

E2. Which model won? The TF-IDF baseline, at 0.775, effectively tied with the AND ensemble at 0.776. The windowed transformer alone scores 0.694 and loses clearly. This is our central finding and we present it as a result, not an embarrassment.

E3. Why did the transformer lose? Did you test the alternatives? Yes, one at a time, and that is the contribution. *Training budget?* No. Retraining on all 53,662 windows instead of 24,000 moved micro-F1 by minus 0.0004. *Seed?* No. The gap of 0.0805 is 9.5 times the transformer’s seed SD of 0.0085, and even the luckiest of three seeds still loses by 0.0665. *Aggregation rule?* No. Six rules, each at its own best threshold, all stay behind. *Threshold?* **Partly, and we say so.** The un-chosen 0.5 costs 0.056 micro-F1; at 0.9 the transformer reaches 0.750. But 0.750 is a threshold tuned on the test set, already a generous upper bound, and it is still below 0.7747. After testing every alternative we could, it loses.

E4. Why would a bag-of-words model beat a transformer at all? Two reasons that work together. Legal drafting is highly formulaic, so distinctive vocabulary carries unusual signal. And the baseline reads the **whole** document while the transformer sees 2,000 characters at a time and has to aggregate, which is exactly where max-pooling injects false positives. The transformer is not being beaten at language understanding, it is being beaten by an architecture that does not have to solve the long-document problem at all.

E5. Is micro-F1 the right metric? No, and this is the most important result in the thesis. Two thirds of the aggregate’s mass is Low-risk categories, dates, party names, governing law. When we split the evaluation by risk band, the AND ensemble, our headline configuration chosen on micro-F1, **misses 37.5 percent of High-risk clauses**. The transformer alone misses 15.9 percent. The permissive OR rule, which finishes **last** on micro-F1 at 0.696, misses only 13.6 percent. The configuration that wins on the aggregate metric is the worst one for the clauses that matter most. A system built to warn a signer was being selected on a metric dominated by the clauses that matter least.

E6. So why did you not just switch to OR? Because choosing it now, after seeing the test set, would be test-set selection, and the whole value of our held-out numbers is that we never did that. The correct procedure is in future work: define a risk-weighted criterion, carve a validation split from the 408 training contracts, select on it, and report once.

E7. What is the end-to-end performance? 21.7 percent, with a 95 percent interval of [19.6, 23.9]. Of 1,348 gold clauses, presence recovers 1,052 (78.0 percent recall), and of those only 293 produce a span that overlaps the gold clause. Component scores of 0.78 and 0.76 would predict roughly 60 percent; we get 21.7.

E8. Where exactly does it fail? We decomposed it. Of the 1,052 detected clauses, the presence model’s best window actually contains the gold clause 68.3 percent of the time. Given a correct window the span model succeeds 39.7 percent of the time. Given a wrong window it succeeds 8 times out of 334, which is near zero as expected. So two separate failures compound: a window-selection failure, the presence model’s **argmax** window is not always the window with the clause, and a span failure caused by a train/inference mismatch, the model was trained on windows re-centred 150 characters before the answer and at inference it sees windows where the answer can be anywhere. Both are fixable without a larger model, which is the useful part of the finding.

E9. Why is span token-F1 0.763 but exact match much lower? Because annotated legal spans are long and their exact boundaries are genuinely debatable, should a leading section number be included? Content overlap is the metric that carries information here. 82.7 percent of predictions overlap the gold span at F1 at or above 0.5.

E10. Are your results comparable to the published CUAD numbers? No, and we never present them as such. Two differences. Their backbone is DeBERTa-v3-large, roughly five times our parameter count. And their protocol is a full-document ranked evaluation reporting precision at 80 percent recall; ours measures extraction within answer-bearing windows. Implementing their protocol is in future work.

E11. Is your model calibrated? No. The max-pooled transformer has Expected Calibration Error 0.201 and a Brier score of 0.194, and it is overconfident across the whole range. Among predictions in the 0.5 to 0.6 bin the clause is actually present 19.5 percent of the time. Even above 0.9, which the interface draws as a nearly full bar, it is present 72.8 percent of the time. A user reading “96 percent confident” is looking at something closer to 73 percent. The cause is max-pooling, taking a maximum over dozens of windows is not a probability-preserving operation and nothing in training asked it to be. The TF-IDF baseline is differently miscalibrated, ECE 0.172, but in the safer direction, it is **under**confident over most of its range.

E12. How does the ensemble do on rare categories? It makes them worse, and we report that. On the ten rarest categories the AND rule nets no F1 gain over the transformer alone, 0.343 against 0.343, buying precision entirely at the cost of recall, and it zeroes out three categories the transformer alone was partly finding. One of those rare categories is High-risk in our taxonomy, so this is a limitation of the deployed system and not just of the leaderboard.

E13. How much uncertainty is there overall? We measured three independent sources and all three exceed the margin we had ranked a leaderboard by. Cluster bootstrap over contracts, about 0.008 half-width. Model seed, SD 0.0022. Dataset split, SD 0.0063 over 20 splits. The ensemble’s margin over the baseline was +0.0014. The lesson we draw, and it generalises: **a margin should be compared against its own noise before it is reported.**

Part F , Risk taxonomy questions

F1. How did you assign the risk levels? Five dimensions, scored from the weaker party’s side: financial exposure and whether it is bounded; irreversibility; restriction of future freedom; asymmetry; and duration or survival past the contract. The rule: **High** if unbounded financially, or irreversible, or it restricts post-term freedom. **Low** if purely informational, a date, a name, a jurisdiction, or it confers a benefit. **Medium** otherwise, a real obligation that is bounded and reversible.

F2. Give an example of a non-obvious assignment. Two good ones. *Cap on Liability* is Medium, not Low, despite sounding protective, because it bounds recovery and can under-compensate a genuine loss. *Governing Law* is Low despite being consequential in litigation, because it is a fact about the contract rather than an obligation falling asymmetrically on the signer.

F3. What are the eight High categories? They fall into three groups by trigger. Money you could owe without limit: Uncapped Liability, Liquidated Damages, Minimum Commitment, Most Favored Nation. Something you cannot get back: IP Ownership Assignment, Irrevocable or Perpetual License. Freedom to earn later: Non-Compete, Exclusivity.

F4. Who is the “weaker party”? **That is vague.** It is, and we define it operationally. The weaker party is the party with less bargaining power over the drafting, which in CUAD is in practice the party that did not write the agreement. Where that is unclear from the category alone we score from the position of the party on whom the obligation falls, so Non-Compete is scored from the side restrained, Audit Rights from the side audited, License Grant from the side receiving. It is a heuristic with a known failure mode: a clause that is High-risk for one side is often protective for the other, so the same badge shown to a licensor and a licensee is not equally informative. The app states the assumption.

F5. Who validated the taxonomy? Nobody. This is the question to answer directly and without flinching. All 41 assignments were made by the three of us. We are computer science students, not lawyers. No legal professional has reviewed them. We state this in the methodology, in the limitations, and in the conclusion, and we prepared the review instrument, `review/risk_taxonomy_review_sheet.csv` plus `review/agreement.py`, so that whoever picks this up can run the review and report the disagreement count whatever it turns out to be.

F6. Why does the lack of validation matter more here than elsewhere? Because every other component is measured against gold annotations experts produced, and this one is measured against nothing. It also has the most direct route to the user: the level decides what Clausify tells a signer to worry about first, so an error here is not a lower score in a table, it is a warning that does not appear.

F7. Why would you report kappa rather than percentage agreement? Because the levels are unevenly distributed, 22 of 41 are Medium. A reviewer who answered “Medium” throughout would score 54 percent agreement while contributing no information at all. Cohen’s kappa for a pair, Fleiss’ for three or more, removes exactly that artefact. The analysis script is written in advance so the result cannot be shaped after the fact.

F8. Is category-level risk good enough? No, and we are careful with the naming because of it. The system does not read a clause and judge how dangerous *that* clause is; it identifies which categories a contract contains and attaches a fixed level to the category. So we say “contracts containing a non-compete deserve attention”, never “this non-compete is unusually harsh”. That is why we call it **category-level risk prioritization** and not risk classification. It cannot distinguish a ten-thousand-dollar liability cap from a ten-million-dollar one. Instance-level severity is in future work.

F9. Does jurisdiction affect the taxonomy? Yes, and it is a threat we name. A non-compete’s enforceability varies by jurisdiction, so a level that is right in one place may be wrong in another. CUAD is entirely US contracts from SEC filings, and the deployment context that motivated this work is Bangladesh, where we have no evidence at all.

Part G , Summarizer questions

G1. What does the summarizer do? Honestly: it does not summarize. It is a classification-to-template component and we describe it that way everywhere in the report.

G2. How do you know? We built the experiment to detect exactly this. We evaluated the fine-tuned FLAN-T5-small on 150 held-out clauses against two reference sets, the 41 category templates and 150 clause-specific references written for those individual clauses. The scores: 0.775 ROUGE-L against templates, 0.116 against clause-specific references. And the decisive row, **the bare category template with no model involved at all scores 0.115**, statistically indistinguishable from the model’s 0.116. A constant lookup table achieves the model’s entire score.

G3. What is it actually emitting? On all 150 test clauses it emits one of the 41 template sentences **verbatim**, 100 percent of outputs are an exact copy of a template, the correct one 70 percent of the time and a different category’s template the other 30 percent. It never produces a sentence of its own. When it is wrong it is wrong the way a classifier is wrong: a Price Restrictions clause about a 10 percent rate cap gets answered with the ROFR template.

G4. Why did this happen? Because of the training targets, not the model. CUAD has no plain-English rewrites, so we wrote 41 templates and reused them across all 11,156 training pairs. Two Cap on Liability clauses with completely different caps share a target string character for character. With 41 unique targets covering thousands of clauses, the loss is minimised by learning to recognise the category and reproduce its sentence. Fine-tuning did not teach it to summarize, it taught it to pick one of 41 strings.

G5. Did fine-tuning help or hurt? Against clause-specific references, zero-shot FLAN-T5-small scores 0.299, more than twice the fine-tuned model’s 0.116. Fine-tuning made it worse at the actual task. We do not claim zero-shot is a good legal summarizer, it largely echoes the source text and echoing inflates a longest-common-subsequence metric. The honest reading is narrower and still uncomfortable: our fine-tuning traded away clause-specific content for fluent category boilerplate that scores well only against a reference set built from the same boilerplate.

G6. What is the broader lesson? An automatic metric measures its reference set, not the task. ROUGE-L 0.775 became 0.116 when the references changed and nothing else did. If we had reported only the first number, which is the number a normal methodology would produce, the thesis would contain a false claim.

G7. How would you fix it? Grow the 41-template seed into a clause-diverse target set, a few thousand clause-specific summaries, and re-fine-tune. Our 150-clause pilot is a working prototype of both the annotation protocol and the evaluation that would detect success. And evaluate for faithfulness rather than overlap: BERTScore, an entailment-based consistency check, readability, and a human rating of whether legal meaning survives.

Part H , Clausify application questions

H1. What is Clausify and what is it built with? A single-page Streamlit web application, `app.py` on top of an inference module `model_utils.py`. It loads the two trained DistilBERT checkpoints, runs them live on a contract the user provides, and returns a risk-prioritized report. All inference on CPU for portability.

H2. Walk me through the user flow. A toggle offers paste-text or upload a `.txt` file, and a confidence slider defaults to 0.50. One click runs both models with a progress bar across the 41 categories. The output is a summary bar counting High, Medium and Low detections, then one card per detected clause showing the category name with a colour-coded risk badge, the one-line plain-English reason, the confidence bar, and the extracted clause text. An expandable table lists all 41 categories with their level and raw score, so nothing the models computed is hidden.

H3. Does the app reproduce the training inference path exactly? Yes, deliberately. Same 2,000 / 1,500 windowing, same CUAD question strings, same max-pooling, same 1,200-character focus chunks for the span model. The only departures are the 30-window cap and the missing TF-IDF leg, both documented below.

H4. How fast is it? Measured through the app's own `analyze()` entry point on test contracts across the length distribution, on an M4, CPU only, models already loaded: shortest contract 2.1 s, 25th percentile 13.3 s, median **27.1 s**, 75th percentile 32.4 s, longest 32.0 s. Mean 21.4 s, peak memory 1.52 GB, cold model load adds 0.4 s.

H5. An earlier version of your report said 5 to 15 seconds. Why the change? Because that was an impression from watching the demo on short inputs, not a measurement, and it was wrong. We benchmarked it and replaced the claim with the table. A median contract takes 27.1 seconds.

H6. Why is latency linear in document length, and why do the two longest buckets take the same time? The dominant cost is 41 categories times the number of windows, forward passes through DistilBERT, and nothing is cached across categories. The 75th-percentile and longest contracts take the same time despite differing fourfold in length because both hit the **30-window cap**.

H7. What does the 30-window cap cost you? It is a latency decision with a silent recall cost, and we quantify it. 30 windows is about 45,500 characters, so any contract longer than that is only partly read and clauses past that point cannot be detected at all. That is not an edge case: **38.4 percent of CUAD contracts, 196 of 510, exceed 45,500 characters**, and the longest test contract is 289,615 characters, of which the app reads the first 16 percent. None of the Chapter 6 numbers reflect this, because they score full documents, so it is a limitation of the deployed demo rather than of the method.

H8. Which model configuration does the app run, and why not the headline ensemble? The transformer alone. The TF-IDF leg needs the fitted vectorizer, which the packaged demo leaves out, so the original reason was packaging. But there is a better reason we discovered afterwards: the AND ensemble misses 37.5 percent of High-risk clauses while the transformer alone misses 15.9 percent. The demo's configuration is worse on aggregate micro-F1 and better at surfacing the clauses a signer would most regret missing. We record it as an accident rather than a design decision, but it is the configuration we would now choose deliberately.

H9. The confidence bar , can a user trust it? No, and the app should say so more loudly than it does. ECE is 0.201 and the model is overconfident throughout: above 0.9, a nearly full bar, the clause is present about 73 percent of the time; in the 0.5 to 0.6 band, under 20 percent. Moving the slider does trade recall for precision in the right direction, but the number on it should not be read as a probability. Fitting a calibration map on a validation split would fix this without retraining and is the cheapest available improvement to the interface.

H10. Can a user trust the quoted clause text? Less than the badge above it. End-to-end survival is 21.7 percent, so the quoted text overlaps the real clause in roughly a fifth of cases when the window is chosen by the presence model rather than handed over. The honest instruction, and it is in the app, is: **the risk badge and the category name are much more reliable than the quoted sentence beneath**

them. A card should be read as “this contract appears to contain a clause of this type, look here”, not as “this is the clause”.

H11. What about privacy? The user is pasting a contract. Everything runs locally. The models are local checkpoints, inference is CPU in-process, nothing is transmitted to any remote service, and no contract is written to disk by the app. But be precise about where that guarantee comes from: it holds because the demonstration runs locally. Hosted on a shared server the same code would place the user’s contract in the memory of a machine they do not control, and Streamlit’s session model would not by itself stop the operator reading it. The privacy property comes from the deployment topology, not from anything the application does. The rule we would apply to any extension: the moment any part of this pipeline calls a remote service, the privacy story changes completely.

H12. How does the app handle a contract in a format other than .txt? It does not. Input is pasted text or a UTF-8 .txt upload. PDF and DOCX parsing, which real contracts arrive as, is not implemented, and PDF extraction on legal documents is its own non-trivial problem, columns, headers, scanned pages needing OCR.

H13. What are the two test contracts shipped with the app? A genuine CUAD contract, and a synthetic “Master Software License and Distribution Agreement” we wrote specifically to exercise a wide spread of the 41 categories. On the synthetic one the app reports 7 High, 15 Medium and 8 Low, 30 of 41 categories detected, and puts Uncapped Liability, IP Ownership Assignment, Exclusivity and Liquidated Damages at the top, which is exactly the before-you-sign ordering the design aims for.

H14. Why did you write a synthetic contract? Is that not cheating? It is a demo fixture, not an evaluation. No number in the results chapter comes from it. Its purpose is to show all three risk bands populated in a single screenshot, which a random CUAD contract will not do.

H15. How is the app deployed, and could you scale it? It runs locally with `streamlit run app.py`, and a single environment variable `CUAD_MODELS_DIR` redirects it if the checkpoints move. Scaling it is not a research problem but there is a real cost: 41 categories times windows forward passes per contract, with no caching across categories, is the obvious optimisation target. Batching all 41 questions against one window, or sharing the window encoding across categories, would cut the dominant cost substantially.

H16. Has anyone outside your group used it? No. No usability evaluation was carried out. The application’s central claim, that grouping clauses by risk helps a non-lawyer act on a contract, is untested, and we list it under construct validity rather than among the results. The protocol for that study is written and in the repository, `review/USABILITY_PROTOCOL.md`.

H17. What would a usability study need to look for specifically? Over-reliance. Given that the confidence display is miscalibrated and the plain-English sentence is category-generic, the specific risk is that a user treats the quoted clause as authoritative or reads the confidence bar as a probability. Those are the two behaviours the study would need to measure, not just task time.

Part I , Ethics, limitations and threats to validity

I1. Is this giving legal advice? No, and the framing matters. The system prioritises categories for attention; it does not advise. The output is “a clause of this type appears to be present and clauses of this type typically deserve attention”, which is triage, not advice. That said, the risk of a user treating it as advice is real and is precisely what an unrun usability study would need to measure.

I2. What is the worst thing that could happen if someone used this today? A High-risk clause is present, the system misses it, the report shows nothing, and the user signs believing they have been warned. With the AND ensemble that happens for 37.5 percent of High-risk clauses. A false negative here is strictly worse than a false positive, which is the whole argument for selecting on a risk-weighted criterion rather than micro-F1.

I3. State your threats to validity. Four headings. *Internal:* hyperparameters never tuned, which protects the test set but means the transformer competes at defaults; the summarizer’s exploratory run monitored test-split clauses; the 64 percent retrieval ceiling was measured before the split was drawn so it saw test contracts; span and summarizer are single runs. *Construct:* ROUGE-L does not measure summarization here and we prove it; micro-F1 does not measure user harm and we prove it; category-level risk is not clause-level risk; the risk bands are our own unvalidated construct; token-F1 assumes span boundaries are well defined when they are genuinely debatable. *External:* every number comes from 510 US commercial contracts from SEC filings, in English. No evidence about other jurisdictions including Bangladesh, consumer or employment agreements, non-English contracts, or machine-drafted agreements. CUAD’s SEC provenance biases toward large-corporate agreements between represented parties, close to the opposite of the small signers we aim at. *Statistical:* three variance sources measured and they disagree in magnitude; three seeds is a small sample for an SD; five categories have at most seven test positives so their per-category F1 is not an estimate; one split, one corpus.

I4. What are the biggest limitations of the artefact itself? No usability evaluation. Downsized backbones. Single-seed, single-budget training for span and summarizer. The rare tail is close to unlearnable at this dataset size and the ensemble makes it worse. Span evaluation scope is narrower than CUAD’s protocol. The summarizer does not summarize. The risk taxonomy is unvalidated and per-category. One dataset, one jurisdiction.

I5. Was any part of this work assisted by AI tools, and how do you declare it? Yes, in one declared place that affects a reported number: the 150 clause-specific reference summaries in the summarizer pilot were drafted with a large-language-model assistant working from clause text alone, with no sight of any model output. We label them a pilot reference set, not lawyer-authored annotation, and we make no claim beyond that.

I6. Is the work reproducible? Yes, and it was a design goal. Every artefact is saved: the split, the fitted baseline, the three model checkpoints. A standalone evaluation notebook regenerates every number in the results chapter from those files alone, and `analysis/verify_numbers.py` mechanically cross-checks the figures quoted in the text against the result JSON files.

Part J , Hard and hostile questions

J1. “Your transformer lost to TF-IDF. So what did you actually contribute?” The contribution is not the leaderboard, it is what we established about it. Three negative results, each of which took an experiment to earn: the windowed transformer does not lose for budget, seed, pooling or threshold reasons, we tested all four; the summarizer does not summarize and a constant lookup table matches its score; and the pipeline loses most of what its parts find, 21.7 percent against about 64 percent predicted, localised to two fixable causes. Plus the result we did not expect, that our evaluation metric and our stated purpose pointed in opposite directions. A thesis that reports a 0.78 and stops would have known none of that.

J2. “Isn’t 0.776 versus 0.775 just noise? Why is it in your abstract as a headline?” It is noise, and we are the ones who proved it. The bootstrap interval is $[-0.0068, +0.0089]$ and the ensemble leads in 62.6 percent of resamples. We report the top two rows as a tie, in the text, and the methodological lesson we draw from it is that a margin should be compared against its own noise before it is reported. We measured three noise sources and all three exceed the $+0.0014$ margin.

J3. “You wrote the risk taxonomy yourselves and you are not lawyers. Why should anyone take it seriously?” They should take it seriously as an auditable design artefact and not as a validated one, and that is exactly how we describe it. Its virtue is that every assignment is written down with a stated reason against five named criteria, so a practitioner can disagree with a specific assignment on specific grounds, which is not true of a learned score. We prepared the review instrument, we specified the statistic in advance, and we report, in the methodology, the limitations and the conclusion, that the review has not been carried out and that no number from it appears anywhere in the thesis.

J4. “Your end-to-end number is 21.7 percent. That means the system does not work.” As an end-to-end clause-quoting system, correct, and we say so. What works is the first stage: presence detection at 78 percent recall, which is what drives the risk report. The badge and the category are reliable, the quoted sentence is not. And the 21.7 is diagnostic rather than final, we decomposed it to 68.3 percent window selection and 39.7 percent span success given a correct window, and both causes have named fixes that need no bigger model.

J5. “Why should we believe your numbers at all?” Because the procedure is hostile to us rather than friendly. Contract-level split so no clause leaks. No hyperparameter tuning at all, so nothing was fitted to the test set. Parameter-free ensemble rules. Cluster bootstrap rather than the narrower naive interval. And we report the one place we touched test data, the summarizer’s exploratory run, along with why no reported number depends on it. Every number is regenerable from saved artefacts and mechanically cross-checked.

J6. “You claimed Longformer would not fit in memory and that turned out to be false. What else is wrong?” That is a fair hit and we found it ourselves. We asserted an impossibility that turned out to be a tight fit, 14.71 GB on a 16 GB machine at batch size 1, and we left the original reasoning in the report with the correction attached where the measurement is. The lesson we drew and wrote down is that feasibility ceilings are cheap to measure and expensive to assume. The same discipline is what produced the 64 percent retrieval ceiling, which saved a wasted training run, and the latency table, which corrected our own “5 to 15 seconds” claim.

J7. “This only works on American contracts.” Correct, and it is the external-validity threat we name first. Every number comes from 510 US commercial contracts from SEC filings. We have not evaluated a single contract from outside that corpus, including from Bangladesh, which is the context that motivated the work. The cheapest honest first step is in future work: run even three locally sourced English-language commercial contracts through the pipeline and inspect the output by hand. That would tell us more about generalisation than any further tuning on CUAD.

J8. “If you started again tomorrow, what would you do differently?” Four things. Draw the split before any measurement, including the retrieval ceiling. Carve a validation split out of the training contracts at the start, so threshold, pooling rule and ensemble rule could be selected legitimately. Define the evaluation metric from the stated purpose, risk-weighted, before running anything, rather than discovering at the end that micro-F1 and our purpose disagree. And get the taxonomy reviewed in week one, when there was still time to act on the disagreements.

J9. “What is the one sentence you want us to remember?” Component scores do not compose. Two stages at roughly 0.78 and 0.76 produced an end-to-end rate of 21.7 percent, and the configuration that won on our aggregate metric was the worst one for the clauses our system exists to warn people about.

Part K , Number sheet

Dataset

Source	CUAD v1, Atticus Project, NeurIPS 2021 D&B, HF theatticusproject/cuad
Contracts / categories	510 / 41
Labelled rows	20,910, 32.1% positive
Split	408 train / 102 test, contract-level, seed 42
Split rows	16,728 (32.0%) / 4,182 (32.2%), Pearson r = 0.986
Contract length	median 33,143 chars, max 338,211
Clause length	median 196 chars, p90 587, p99 1,330
Rarest / commonest	Source Code Escrow 2.5% (13 contracts) / Document Name 100%
Annotation value	approx. \$2M of attorney time

Models and training

Baseline	TF-IDF 20k features, 1-2 grams, min_df 2 + 41 balanced logistic regressions
Presence	DistilBERT-base seq-cls, 2,000/1,500 windows, max-pool, 24,000 of 53,662, 256 tok, 2 ep, bs 16, lr 2e-5, MPS, ~50 min
Span	DistilBERT-base QA, 1,200-char focus window at +150, 8,000 of 11,156, 320 tok, 2 ep, bs 16, lr 3e-5, ~23 min
Summarizer	FLAN-T5-small, 41 templates, 4,000 of 11,156, 256 in / 48 out, bs 8, lr 3e-4, CPU, ~8 min
Optimizer	AdamW, linear decay, no warmup, clip 1.0 (98.4% of steps clipped)
Hardware	Apple M4, 16 GB unified, torch 2.8, transformers 5.x

Presence results (4,182 test decisions)

Model	micro-F1
AND ensemble	0.776
TF-IDF baseline	0.775
AVG ensemble	0.718
OR ensemble	0.696
Transformer (max-pool)	0.694
Transformer on all 53,662 windows	0.694
AND on that transformer	0.782

Accuracy 85.5%. Macro-F1 0.572 (41 cats) / 0.666 (34 cats with ≥ 10 positives). Overall precision / recall: TF-IDF 0.742 / 0.810, transformer 0.562 / 0.908.

Split by category frequency (precision / recall / F1)

Group	Transformer	TF-IDF	AND
Rarest 10 (70 positives)	0.261 / 0.500 / 0.343	0.429 / 0.300 / 0.353	0.586 / 0.243 / 0.343
Other 31 (1,278 positives)	0.582 / 0.930 / 0.716	0.753 / 0.838 / 0.793	0.776 / 0.810 / 0.792

Variance and robustness

Bootstrap, ensemble vs baseline	[-0.0068, +0.0089], ahead in 62.6% of resamples , a tie
Seed SD	transformer 0.0085, AND 0.0022; gap = 9.5 SD
Split SD	0.0063 over 20 splits (mean 0.7829, ours 0.7747)
Full data (53,662 windows)	micro-F1 change -0.0004
Pooling ablation	6 rules; best cell max@0.9 = 0.7504, still < 0.7747
Threshold	transformer 0.694@0.5 -> 0.750@0.9 (+0.056)
Calibration	ECE 0.201, Brier 0.194; >0.9 bin actually 72.8% present; TF-IDF ECE 0.172
Retrieval ceiling	64.0% (category name, top-3)

Risk-band results (High = 8 categories, 176 test positives)

Rule	High recall	High missed
OR	0.864	13.6%
Transformer	0.841	15.9%
TF-IDF	0.648	35.2%
AND (headline)	0.625	37.5%

Span and end-to-end

Token-F1	0.763, cluster CI [0.740, 0.782], naive CI [0.751, 0.774]
Overlap at F1 >= 0.5	82.7% of 2,667 answer-bearing windows
End-to-end	21.7% [19.6, 23.9]; 1,348 gold -> 1,052 presence -> 293 span
Decomposition	window correct 68.3%; span ok given correct window 39.7%; given wrong window 8/334

Summarizer

ROUGE-L vs 41 templates	0.775
ROUGE-L vs clause-specific	0.116
Zero-shot vs clause-specific	0.299
Bare template, no model	0.115
Verbatim template output	100% (correct template 70%)

Longformer benchmark (Apple M4, 16 GB)

Forward, 4,096 tok, bs 1	3.69 s, 2.18 GB
Training step, bs 1	258 s, 14.71 GB
Training step, bs 2	2,170 s, 20.75 GB (paging)
DistilBERT step, 256 tok, bs 16	3.17 s, 4.47 GB
Our 2-epoch schedule at that rate	approx. 136 days
Inference, one median contract	10.6 s DistilBERT vs 454 s Longformer

Clausify application

Stack	Streamlit, CPU inference, 2 DistilBERT checkpoints
Latency	median 27.1 s, mean 21.4 s, worst 32.4 s, cold load +0.4 s
Memory	peak RSS 1.52 GB
Window cap	30 windows = approx. 45,500 chars; 38.4% of CUAD (196/510) exceed it
Configuration	transformer alone (no TF-IDF leg)
Risk taxonomy	8 High / 22 Medium / 11 Low
Demo output (synthetic contract)	7 High, 15 Medium, 8 Low; 30 of 41 detected
